

OPERATING SYSTEMS PROJECT  
REVIEW 2 — PROJECT REPORT

# Event-Driven Runtime-Adaptive CPU Scheduling

## with Sequential Tabular Q-Learning

Single-CPU discrete-event simulation • causal arrived-work state abstraction • undiscounted waiting-time learning • predeclared heuristic baseline • held-out crossed-bootstrap evaluation

| Field                   | Value                                                                   |
|-------------------------|-------------------------------------------------------------------------|
| Project title           | Event-Driven Runtime-Adaptive CPU Scheduling with Sequential Q-Learning |
| Review stage            | Review 2 (implementation complete, experiments verified)                |
| Repository              | github.com/lostpilot404/OS-Project                                      |
| Implementation language | Python 3.11.2 (NumPy, pandas, Matplotlib, pytest)                       |
| Automated verification  | 273 checks, all passing (re-run during this build)                      |
| Experiment status       | Deterministic and re-run verified from committed seeds                  |
| Report generated        | 10 October 2026                                                         |

|                  |                         |                    |                         |                         |                 |
|------------------|-------------------------|--------------------|-------------------------|-------------------------|-----------------|
| 6                | 162 × 4                 | 6,000              | 210                     | 5                       | 273             |
| methods compared | causal states × actions | training workloads | held-out test workloads | independent model seeds | automated tests |

**Abstract.** This report documents Review 2 of an operating-systems project that replaces a fixed CPU scheduling policy with a causal, event-driven runtime controller. A single-CPU discrete-event simulator keeps one persistent FIFO ready queue and, at every dispatch or Round-Robin quantum boundary, asks a controller to choose FCFS, SJF, Round Robin or Priority for the next execution segment of the same evolving trace. The controller observes only work that has already arrived: five causal features (ready-queue size, completed count, median ready remaining burst, mean ready arrival age, and arrived-work priority spread) are discretised into 162 states, and a tabular Q-learning agent with an undiscounted interval reward  $r_t = -\Delta W_t/q$  ( $\gamma = 1$ , so  $\sum r_t = -\text{total waiting time} / q$ ) learns a switching policy over 6,000 seeded training workloads across five independent seeds. On 210 untouched held-out workloads (master seed 19301, quantum 4, switch cost 1) the learned controller averages 104.789 time units of waiting time against 127.797 for FCFS, 215.729 for Round Robin and 203.404 for a predeclared non-RL heuristic, while fixed SJF — which is non-preemptive and pays no mid-job switch cost — remains lowest at 93.345. The report states that negative result explicitly, quantifies every comparison with family-stratified crossed-bootstrap confidence intervals, and documents the simulator's assumptions and limits.

**Keywords:** CPU scheduling; discrete-event simulation; reinforcement learning; Q-learning; causal state abstraction; Round Robin; SJF; bootstrap confidence intervals.

# Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Contents</b>                                                | <b>1</b>  |
| <b>1. Introduction</b>                                         | <b>3</b>  |
| 1.1 Problem statement . . . . .                                | 3         |
| 1.2 Motivation . . . . .                                       | 3         |
| 1.3 Objectives . . . . .                                       | 3         |
| 1.4 Scope, assumptions and non-goals . . . . .                 | 3         |
| <b>2. Literature survey</b>                                    | <b>4</b>  |
| <b>3. Requirement analysis</b>                                 | <b>5</b>  |
| 3.1 Functional requirements . . . . .                          | 5         |
| 3.2 Non-functional requirements . . . . .                      | 5         |
| 3.3 Software and hardware requirements . . . . .               | 5         |
| <b>4. System design</b>                                        | <b>6</b>  |
| 4.1 Architecture . . . . .                                     | 6         |
| 4.2 Execution semantics and the ready-queue contract . . . . . | 6         |
| 4.3 The causal observation and one decision epoch . . . . .    | 7         |
| 4.4 Causal state abstraction . . . . .                         | 7         |
| 4.5 Learning objective and update rule . . . . .               | 8         |
| 4.6 Predeclared non-RL heuristic baseline . . . . .            | 8         |
| <b>5. Algorithms</b>                                           | <b>9</b>  |
| 5.1 Runtime decision loop . . . . .                            | 9         |
| 5.2 Action (fixed-policy) selection rules . . . . .            | 9         |
| 5.3 Sequential Q-learning controller . . . . .                 | 9         |
| <b>6. Implementation</b>                                       | <b>10</b> |
| 6.1 Command-line interface . . . . .                           | 10        |
| 6.2 Verified run output . . . . .                              | 10        |
| 6.3 Determinism and split integrity . . . . .                  | 10        |
| <b>7. Experimental setup</b>                                   | <b>11</b> |
| 7.1 Workload families . . . . .                                | 11        |
| 7.2 Splits and protocol . . . . .                              | 11        |
| 7.3 Configuration . . . . .                                    | 11        |
| 7.4 Metrics . . . . .                                          | 11        |
| <b>8. Results and discussion</b>                               | <b>12</b> |
| 8.1 Held-out final test . . . . .                              | 12        |
| 8.2 Spread across workloads . . . . .                          | 12        |
| 8.3 Per-family behaviour . . . . .                             | 13        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 8.4 Learning dynamics . . . . .                            | 14        |
| 8.5 Paired comparisons with uncertainty . . . . .          | 14        |
| 8.6 Key findings . . . . .                                 | 15        |
| 8.7 Tabular coverage and fallback behaviour . . . . .      | 16        |
| 8.8 Illustrative learned trace . . . . .                   | 16        |
| 8.9 Validation versus held-out test . . . . .              | 16        |
| <b>9. Testing and verification</b>                         | <b>18</b> |
| 9.1 Verification claims covered by the suite . . . . .     | 18        |
| 9.2 Test run output . . . . .                              | 18        |
| <b>10. Legacy offline policy selector (reference only)</b> | <b>19</b> |
| <b>11. Limitations and future work</b>                     | <b>20</b> |
| <b>12. Conclusion</b>                                      | <b>20</b> |
| <b>References</b>                                          | <b>21</b> |
| <b>Appendix A. Reproducibility and artifact index</b>      | <b>22</b> |
| A.1 Commands . . . . .                                     | 22        |
| A.2 Artifact index . . . . .                               | 22        |
| A.3 Environment used for the recorded run . . . . .        | 22        |

# 1. Introduction

## 1.1 Problem statement

A general-purpose operating system must decide which ready process runs next on a CPU. The classical policies each optimise a different, largely conflicting objective: First-Come First-Served (FCFS) is simple and fair in arrival order but suffers the convoy effect; non-preemptive Shortest Job First (SJF) minimises mean waiting time among non-preemptive policies when burst lengths are known, at the cost of response time and starvation risk for long jobs; Round Robin (RR) bounds response time by time-slicing but inflates waiting time and context switches; and static Priority scheduling reflects importance but can starve low-priority work. Real workloads are non-stationary — a trace may begin interactive and become batch-like — so no single fixed policy is best for the whole trace.

The problem addressed by this project is therefore: **can a single-CPU scheduler adapt its policy at run time, at every dispatch or quantum boundary, using only information that a real kernel could actually possess at that instant, and can that adaptation be measured honestly against the fixed policies it switches between?** Review 1 delivered four verified standalone schedulers plus an offline selector that inspected whole-workload summaries; Review 2 replaces that look-ahead design with a causal, event-driven runtime controller and a full experimental evaluation.

## 1.2 Motivation

- **Fixed policies are Pareto-incomplete.** RR buys response time with waiting time; SJF buys waiting time with response time. A runtime controller can trade between them as the ready queue changes.
- **Adaptation must be causal.** A controller that peeks at unarrived processes or at completed counterfactual schedules cannot be deployed and cannot be evaluated fairly. Enforcing a strict observation boundary makes the result meaningful.
- **Learning is a natural fit.** The choice at each epoch is a small sequential decision problem with a delayed, additive cost (waiting time), which maps cleanly onto tabular Q-learning with a 162-state abstraction.
- **Honest negative results matter.** An adaptive controller that fails to beat a strong fixed baseline is a useful, publishable finding — provided the comparison is paired, held out and accompanied by uncertainty intervals.

## 1.3 Objectives

| #  | Objective                                                                                                                                                               |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| O1 | Preserve four verified standalone schedulers (FCFS, SJF, RR, Priority) as baselines.                                                                                    |
| O2 | Build a single-CPU, event-driven runtime environment with one persistent FIFO ready queue, remaining bursts, arrivals, RR re-enqueue and per-process switch accounting. |
| O3 | Define a causal observation contract that exposes only arrived work, and encode it as a compact 162-state tabular abstraction.                                          |
| O4 | Train a sequential Q-learning controller whose undiscounted return equals negative total waiting time, with explicit exploration, masking and unseen-state fallback.    |
| O5 | Declare a non-RL heuristic baseline under the same observation contract, before any test results are seen.                                                              |
| O6 | Evaluate on held-out workloads with paired, family-stratified bootstrap intervals and report negative findings explicitly.                                              |
| O7 | Ship a reproducible, test-covered repository (273 automated checks) with recorded artifacts and regeneration commands.                                                  |

## 1.4 Scope, assumptions and non-goals

- **In scope:** synthetic single-CPU simulation; four fixed policies; one causal runtime controller; one predeclared heuristic; seven workload families; train / validation / held-out-test protocol; uncertainty quantification.
- **Assumption — burst knowledge:** exact CPU burst lengths become known when a process arrives. This matches the standalone SJF simulator already in the project; no burst predictor is claimed.
- **Non-goals:** I/O blocking, multicore and cache/bus contention, priority aging, kernel latency, real Linux integration, and any claim that the measured ordering transfers to production workloads.

## 2. Literature survey

The survey below covers (a) the classical scheduling policies implemented as baselines, (b) heuristic adaptive schedulers used in production kernels, and (c) learning-based scheduling. The gap this project targets is highlighted in the final row: most reported RL schedulers either observe whole-workload/global state that a kernel cannot see at decision time, or they replace the policy entirely instead of switching between classical policies under a declared cost model.

| Area                              | Representative work                                                                                         | What it establishes                                                                                                                          | Gap addressed here                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Classical policies                | FCFS, SJF/SRTF, Round Robin and Priority scheduling as formalised in standard operating-system texts [1][2] | Analytic trade-offs: SJF is optimal for mean waiting time among non-preemptive policies; RR bounds response time; static priority can starve | Used as the four baselines and as the four actions, with a declared quantum and switch cost                                            |
| Heuristic adaptive schedulers     | Multi-level feedback queues and fair-share/virtual-runtime schedulers [1][2]                                | Hand-tuned feedback rules adapt to observed behaviour in real kernels                                                                        | A declared, non-learning heuristic is evaluated as a baseline so that RL is not credited for what fixed rules already achieve          |
| RL for thread/CPU scheduling      | Shrivastava, <i>Reinforcement Learning for Scheduling Threads on a Multi-Core Processor</i> [3]             | Q-learning can learn online from core utilisation / readiness features, but the state space grows very large and convergence is slow         | Keeps the abstraction tiny (162 states, 4 actions) so coverage and fallback behaviour can be measured directly                         |
| Deep RL schedulers                | Deep Q-learning task scheduling [4]; Double DQN OS scheduling [5]                                           | Function approximation handles richer state but needs large data and is hard to audit                                                        | Tabular Q-learning is used precisely so every state-action value, visit count and fallback can be inspected in the committed artifacts |
| Context-aware / learned governors | Context-aware RF + DQN scheduling framework [6]; temporal-encoding DRL for DVFS [7]                         | Workload classification and temporal encoding improve adaptivity on heterogeneous loads                                                      | Workload identity is never given to the controller here; only arrived work is observable                                               |
| Surveys                           | Shyalika, Silva and Karunananda, <i>Reinforcement learning in dynamic task scheduling: a review</i> [8]     | Confirms RL scheduling is dominated by simulation studies with weak baselines and no uncertainty reporting                                   | Paired, family-stratified crossed-bootstrap intervals and an explicit statement of the result that does <i>not</i> favour the learner  |

Two practices from the surveyed literature are adopted deliberately: every learned method is compared against a predeclared rule-based controller sharing the same observation contract, and every headline comparison is reported with a confidence interval rather than a point estimate.

## 3. Requirement analysis

### 3.1 Functional requirements

| ID   | Requirement                                                                                                                                   |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| FR-1 | Generate reproducible synthetic workloads across seven families with seeded RNG.                                                              |
| FR-2 | Simulate a single CPU with arrivals, a persistent FIFO ready queue, remaining bursts and per-process accounting.                              |
| FR-3 | Implement FCFS, SJF, Round Robin and Priority both standalone and as runtime actions, with identical semantics.                               |
| FR-4 | Expose a causal observation containing only arrived/ready work and history counts.                                                            |
| FR-5 | Encode the observation into a bounded tabular state and learn a policy with Q-learning (exploration, masking, fallback).                      |
| FR-6 | Provide a deterministic non-RL heuristic under the same observation contract.                                                                 |
| FR-7 | Charge a context-switch cost whenever the running PID changes, for all methods.                                                               |
| FR-8 | Compute six metrics (waiting, turnaround, response, CPU utilisation, throughput, context switches) and emit CSV/JSON artifacts plus a report. |
| FR-9 | Provide a CLI that reproduces training, validation, held-out testing and the legacy offline reference.                                        |

### 3.2 Non-functional requirements

| ID    | Property        | How it is met                                                                                                |
|-------|-----------------|--------------------------------------------------------------------------------------------------------------|
| NFR-1 | Reproducibility | Every artifact is regenerated from declared seeds; split fingerprint overlap is asserted to be empty.        |
| NFR-2 | Causality       | Controller and encoder APIs accept observations, never workloads; hidden-future invariance is test-enforced. |
| NFR-3 | Determinism     | Evaluation is greedy and read-only; no Q-table or visit-count mutation during evaluation.                    |
| NFR-4 | Auditability    | Q values, visit counts, coverage and fallback counts are written to committed CSV files.                     |
| NFR-5 | Performance     | Controller overhead is measured separately from simulated time (tens of microseconds per decision).          |
| NFR-6 | Testability     | 273 automated checks cover schedulers, encoding, learning, causality and the experiment pipeline.            |

### 3.3 Software and hardware requirements

| Component               | Requirement                            | Used for                          |
|-------------------------|----------------------------------------|-----------------------------------|
| Operating system        | Linux (developed on Linux 6.1, x86-64) | Runs the simulator and tests      |
| Language                | Python 3.11.2                          | Whole implementation              |
| NumPy 2.4.6             | Numerical arrays and seeded RNG        | Q table, bootstrap resampling     |
| pandas 3.0.6            | Tabular result handling                | Metric aggregation and CSV export |
| Matplotlib 3.11.2       | Figure rendering                       | Report and legacy figures         |
| pytest 9.1.1            | Automated verification                 | 273 unit and integration tests    |
| ReportLab (report only) | PDF generation                         | tools/build_review2_report.py     |
| Memory / disk           | < 512 MB RAM, < 50 MB for artifacts    | Full experiment re-run and report |

## 4. System design

### 4.1 Architecture

The runtime system is a closed loop between a discrete-event environment that owns the future and a controller that may see only the past and the present. The environment privately holds the complete event list so it can admit future arrivals at their arrival times; the controller receives an immutable `RuntimeObservation` and returns one of four actions. The action changes only the next selection rule and service length — it never rebuilds the ready queue or resets remaining bursts.

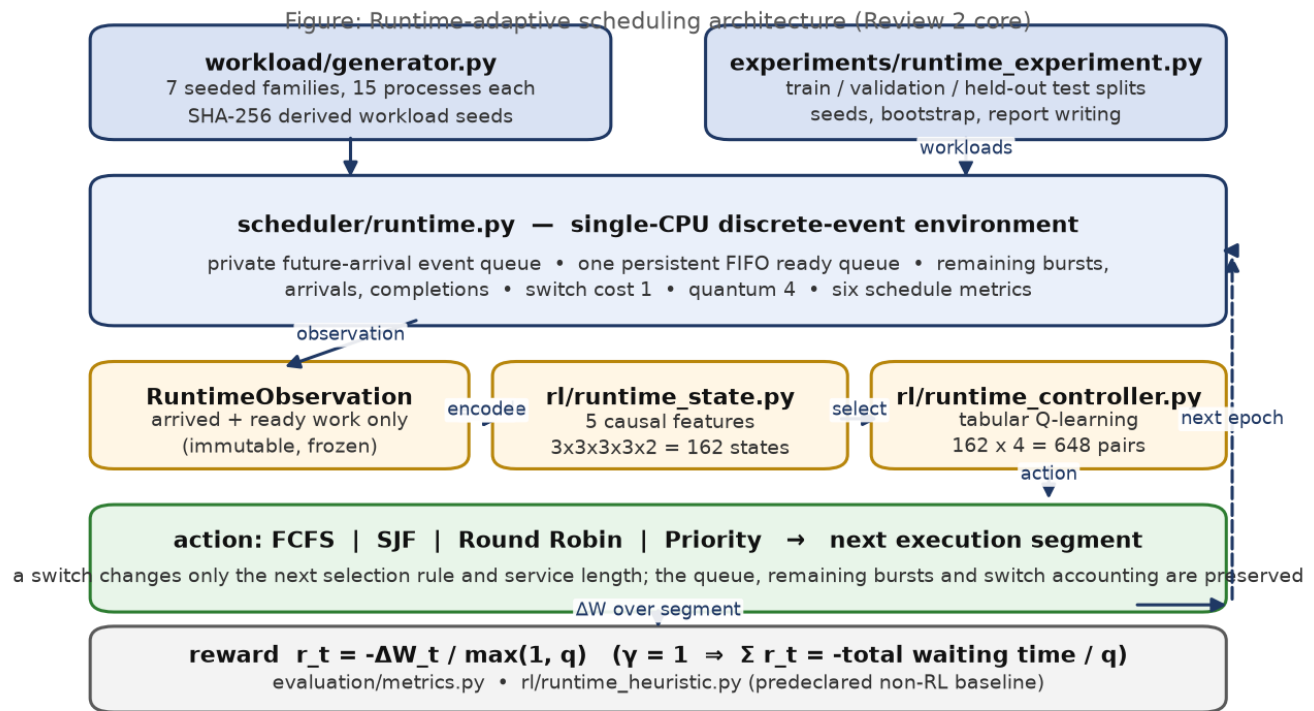


Figure 4.1: Runtime-adaptive scheduling architecture. The controller path (amber) receives only arrived work; the environment path (blue) owns future arrivals and switch accounting.

| Package      | Key files                                    | Responsibility                                                                                  |
|--------------|----------------------------------------------|-------------------------------------------------------------------------------------------------|
| workload/    | models.py, generator.py                      | Process/workload models and the seven seeded workload families; schedule-trace validation.      |
| scheduler/   | runtime.py                                   | Single-CPU discrete-event environment: event queue, FIFO ready queue, switch cost, six metrics. |
| scheduler/   | fcfs.py, sjf.py, round_robin.py, priority.py | Standalone fixed-policy baselines with preserved semantics.                                     |
| rl/          | runtime_state.py                             | Five-feature causal encoder: $3 \times 3 \times 3 \times 3 \times 2 = 162$ states.              |
| rl/          | runtime_controller.py                        | Sequential tabular Q-learning: visits, masking, deterministic fallback.                         |
| rl/          | runtime_heuristic.py                         | Predeclared non-RL adaptive rule under the same contract.                                       |
| evaluation/  | metrics.py, comparison.py                    | Metric definitions and paired/bootstrap comparison helpers.                                     |
| experiments/ | runtime_config.py, runtime_experiment.py     | Frozen seeds, split sizes, switch cost, Q settings; training, validation, testing, reporting.   |
| tests/       | 16 modules                                   | 273 automated checks (schedulers, causality, learning, pipeline).                               |
| tools/       | build_review2_report.py, review2_figures.py  | Regenerates this PDF from the committed artifacts.                                              |

Table 4.1: Module inventory. The legacy offline selector (`rl/adaptive.py`, `experiments/run_experiment.py`) is preserved but is not part of the runtime design (Section 10).

### 4.2 Execution semantics and the ready-queue contract

- One persistent FIFO ready queue holds arrived, unfinished, non-running processes. Arrivals are appended in (arrival\_time, pid) order; dispatch removes the selected process; completion removes it permanently.

- **FCFS** dispatches the current queue head non-preemptively — after an RR requeue this is live FIFO order, not the original arrival time.
- **Round Robin** dispatches the head for  $\min(\text{quantum}, \text{remaining\_burst})$ ; on quantum expiry, arrivals at that endpoint are admitted first and the yielded process is appended last.
- **SJF** picks the smallest remaining burst non-preemptively; ties preserve current ready-queue order.
- **Priority** picks the highest static priority (lower numeric value = higher priority); ties preserve ready-queue order.
- A switch cost of 1 time unit is charged only when the running PID changes. The first dispatch and a redispach of the same PID are free. The cost applies to every method, including the fixed policies.

### 4.3 The causal observation and one decision epoch

At a decision epoch the environment admits arrivals, freezes an observation, asks for an action, executes the resulting segment and returns the waiting-time increment incurred. The observation contains current time and the previous policy; the ready processes with their arrival time, known burst, remaining burst and priority; arrived and completed counts; aggregates over arrived work only (mean burst, priority spread); and the mean arrival age of currently ready jobs. Unarrived process data, the total workload size and counterfactual schedule metrics are never passed to the controller.

Figure: One sequential decision epoch

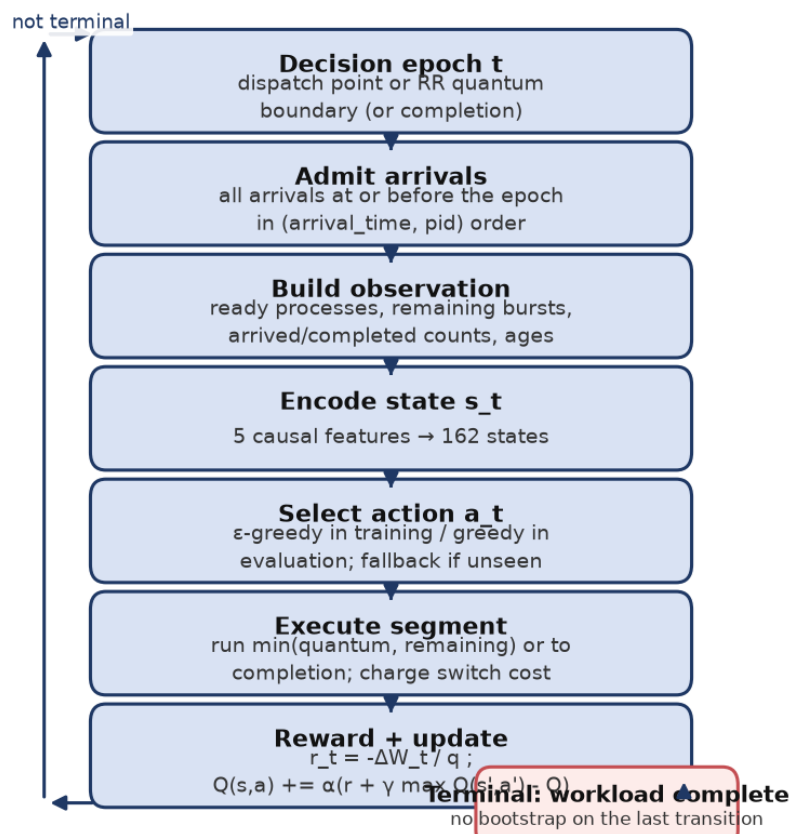


Figure 4.2: One sequential decision epoch. The loop repeats until the workload completes; the final transition omits the bootstrap term.

### 4.4 Causal state abstraction

| # | Feature                      | Bins | Discretisation ( $q = 4$ ) | Causal rationale                                                              |
|---|------------------------------|------|----------------------------|-------------------------------------------------------------------------------|
| 1 | ready_count                  | 3    | {1}, {2–3}, {4+}           | Instantaneous ready-queue contention.                                         |
| 2 | completed_count              | 3    | {0}, {1–3}, {4+}           | Trace progression without knowing how many jobs remain.                       |
| 3 | median_ready_remaining_burst | 3    | $\leq q, \leq 4q, > 4q$    | Remaining service demand of currently ready jobs.                             |
| 4 | mean_ready_arrival_age       | 3    | $\leq q, \leq 4q, > 4q$    | Mean (current time – arrival time) of ready jobs; includes prior CPU service. |

| # | Feature                      | Bins | Discretisation (q = 4) | Causal rationale                                    |
|---|------------------------------|------|------------------------|-----------------------------------------------------|
| 5 | arrived_work_priority_spread | 2    | zero, non-zero         | Whether arrived processes have distinct priorities. |

Table 4.2:  $3 \times 3 \times 3 \times 3 \times 2 = 162$  causal states,  $162 \times 4 = 648$  state-action pairs (rl/runtime\_state.py).

#### 4.5 Learning objective and update rule

Each trace is one sequential episode. Between consecutive epochs the environment measures the actual waiting-time area  $\Delta W_t$  accumulated by ready processes (including any switch interval) and returns a scaled interval reward:

$$r_t = - \Delta W_t / \max(1, q) \quad \gamma = 1.0 \Rightarrow \sum_t r_t = - \text{TotalWaitingTime} / q$$

Because  $\gamma = 1$  and the episode is finite, the undiscounted episodic return is exactly the negative total waiting time divided by the quantum: maximising return is identical to minimising total waiting time. The tabular update is

$$Q(s, a) \leftarrow Q(s, a) + \alpha [ r_t + \gamma \max_{a' \in \text{visited}(s')} Q(s', a') - Q(s, a) ], \quad \alpha = 0.1$$

Terminal transitions omit the bootstrap. The first visit to a state forces a uniform random action; greedy selection and bootstrapping mask unvisited actions; and a wholly unseen evaluation state triggers an explicit fallback to the causal heuristic rather than a silent zero-Q tie.

#### 4.6 Predeclared non-RL heuristic baseline

To separate 'learning helped' from 'any adaptive rule helps', a fixed deterministic rule was declared before any test results were inspected. It sees the same observation:

- **R1.** Round Robin if at least three processes are ready and their mean arrival age is at least one quantum.
- **R2.** Otherwise Priority if at least two ready processes have different priorities.
- **R3.** Otherwise SJF if the largest visible remaining burst is at least twice the smallest.
- **R4.** Otherwise FCFS.

## 5. Algorithms

### 5.1 Runtime decision loop

```

RUNTIME_SCHEDULE(workload, controller, quantum q, switch_cost c):
  ready <- {} # persistent FIFO of arrived, unfinished processes
  events <- sorted arrivals of workload # private to the environment
  t <- 0 ; running <- NULL ; previous_policy <- None
  while unfinished processes remain:
    admit all arrivals with arrival_time <= t # (arrival_time, pid) order
    if ready is empty: # CPU idle
      t <- next arrival time ; continue
    obs <- Observation(t, ready, arrived_count, completed_count, previous_policy)
    a <- controller.select(obs) # FCFS | SJF | RR | Priority
    pid <- SELECT(ready, a) # per policy rule, ties by FIFO order
    if running != NULL and pid != running:
      charge_switch_cost(c) ; t <- t + c ; admit arrivals during the switch
    run <- min(q, remaining[pid]) if a == RR else remaining[pid]
    execute pid for run ; t <- t + run ; remaining[pid] <- remaining[pid] - run
    dW <- waiting-time area accumulated by ready processes during this segment
    controller.observe_reward(-dW / max(1, q))
    settle endpoint: admit arrivals at t; if remaining[pid] > 0 and a == RR: append pid last
                     else if remaining[pid] == 0: mark completed
    previous_policy <- a
  return metrics(waiting, turnaround, response, utilisation, throughput, switches)

```

### 5.2 Action (fixed-policy) selection rules

```

SELECT(ready, a):
  FCFS    -> head of the ready queue (non-preemptive)
  SJF     -> argmin remaining_burst, ties by ready order (non-preemptive)
  Priority -> argmin priority value, ties by ready order (non-preemptive)
  Round Robin -> head of the ready queue, served for min(quantum, remaining_burst)

```

### 5.3 Sequential Q-learning controller

```

SELECT_ACTION(obs): # training: epsilon-greedy; evaluation: greedy + fallback
  s <- encode(obs) # 5 causal features -> [0, 162)
  if evaluating:
    if no visited action in s: return HEURISTIC_FALLBACK(obs) # counted and reported
    return argmax over visited actions of Q[s, a]
  if first_visit(s) or rand() < epsilon(s): return uniform_random_action()
  return argmax over visited actions of Q[s, a]

UPDATE(s, a, r, s', terminal):
  Q[s, a] <- Q[s, a] + alpha * (r + (0 if terminal else gamma * max_visited Q[s', .]) - Q[s, a])
  visits[s, a] <- visits[s, a] + 1

```

Exploration uses an episode-level epsilon schedule; the bootstrap maximum is taken over visited actions only, so unvisited actions can never be credited optimistically.

## 6. Implementation

The simulator, baselines, controller and experiment pipeline are pure Python with NumPy arrays for the Q table and pandas for artifact export. Nothing in the controller path touches the private event list: the observation object is frozen and the encoder accepts only that object, which is what makes the causality tests meaningful rather than cosmetic. Controller wall time is measured separately (observation, selection, Q update and simulator time are distinct columns) and is never charged to simulated time.

### 6.1 Command-line interface

```
python main.py runtime-experiment      # causal train / validate / held-out test
python main.py runtime-experiment --results-dir /tmp/os-project-runtime
python main.py experiment               # legacy offline whole-workload selector
python main.py train                   # train the legacy selector only
```

### 6.2 Verified run output

The block below is the tail of a full reproduction run performed on the reporting host before this PDF was generated; it matches the committed artifacts exactly.

```
$ python main.py runtime-experiment --results-dir /tmp/os-project-runtime
Causal event-driven runtime experiment
-----
training seeds           : 7101, 7102, 7103, 7104, 7105
training episodes/model  : 1200
validation workloads     : 140
final held-out test workloads : 210
final-test mean waiting time:
  FCFS                   : 127.797
  SJF                     : 93.345
  Round Robin            : 215.729
  Priority                : 129.207
  Causal heuristic       : 203.404
  Runtime Q-learning     : 104.789
learned within-trace switches : 15 (23 decisions; runtime_learned_switch_demo.json)
artifacts:
  training_metrics       : runtime_training_metrics.csv
  validation_metrics     : runtime_validation_metrics.csv
  final_test_metrics     : runtime_final_test_metrics.csv
  paired_comparisons     : runtime_paired_comparisons.csv
  state_action_coverage  : runtime_state_action_coverage.csv
  q_table                : runtime_q_table.csv
  workload_manifest      : runtime_workload_manifest.csv
  learned_switch_demo    : runtime_learned_switch_demo.json
  summary                : runtime_summary.json
  report                 : runtime_report.md
```

### 6.3 Determinism and split integrity

Training, validation and test workloads are derived from declared master seeds with SHA-256 workload fingerprints. The pipeline asserts that the union of training fingerprints is disjoint from the validation and final-test sets (`split_fingerprint_overlap = false`; 6,000 distinct training fingerprints). Evaluation runs are greedy and read-only, so results are reproducible bit-for-bit from the recorded configuration.

## 7. Experimental setup

### 7.1 Workload families

| Family                | Description                                                                                       | Burst range | Arrivals  | Priority |
|-----------------------|---------------------------------------------------------------------------------------------------|-------------|-----------|----------|
| short_jobs            | Short-job-heavy: 80% of bursts drawn from 1-5 time units.                                         | 1–50        | uniform   | 1–5      |
| long_jobs             | Long-job-heavy: 20% of bursts drawn from 1-5 time units.                                          | 1–50        | uniform   | 1–5      |
| mixed                 | Uniform mix of bursts over the full 1-50 range.                                                   | 1–50        | uniform   | 1–5      |
| cpu_bursty            | Long CPU bursts (20-50) with dense uniform arrivals.                                              | 20–50       | uniform   | 1–5      |
| priority_skewed       | Uniform bursts; 70% of priorities lie in the 1-2 band.                                            | 1–50        | uniform   | 1–5      |
| staggered_interactive | One long process at time zero plus 14 short processes arriving at integer gaps of 4-6 time units. | 25–50       | staggered | 1–1      |
| poisson_arrivals      | Held-out evaluation condition: Poisson arrivals (rate 0.5) and uniform bursts from 1-20.          | 1–20        | poisson   | 1–5      |

Table 7.1: Seven synthetic families, 15 processes per workload. *poisson\_arrivals* is held out of training entirely.

### 7.2 Splits and protocol

| Split      | Composition                                         | Size                                                  | Seeding           |
|------------|-----------------------------------------------------|-------------------------------------------------------|-------------------|
| Training   | 6 families ( <i>poisson_arrivals</i> excluded)      | 1,200 sequential episodes × 5 seeds = 6,000 workloads | Seeds 7101–7105   |
| Validation | 7 families × 20 repetitions                         | 140 workloads                                         | Master seed 8201  |
| Final test | 7 families × 30 repetitions                         | 210 workloads (untouched)                             | Master seed 19301 |
| Bootstrap  | Family-stratified, workload + model-seed resampling | 1,000 replicates                                      | Seed 104729       |

Table 7.2: Three-way split. Validation is used for reporting and for a predeclared illustrative-demo rule only — never for tuning.

### 7.3 Configuration

| Parameter                | Value                           | Notes                                                      |
|--------------------------|---------------------------------|------------------------------------------------------------|
| Round-Robin quantum      | 4                               | Time slice for the RR action and RR baseline               |
| Context-switch cost      | 1                               | Charged whenever the running PID changes, for every method |
| Priority convention      | lower value = higher priority   | Static priorities, no aging                                |
| Learning rate $\alpha$   | 0.1                             | Tabular Q-learning step size                               |
| Discount factor $\gamma$ | 1.0                             | Undiscounted; $\Sigma r$ = –total waiting time / $q$       |
| $\epsilon$ schedule      | 1.0 → 0.05 (×0.997 per episode) | Episode-level exploration decay                            |
| Initial Q value          | 0.0                             | Neutral; unvisited actions are masked, not assumed optimal |
| State space              | 162 states                      | 5 causal features, 648 state-action pairs                  |
| Burst information        | known on arrival                | Matches the standalone SJF simulator assumption            |

### 7.4 Metrics

| Metric              | Definition                                                        |
|---------------------|-------------------------------------------------------------------|
| avg_waiting_time    | Mean over processes of (start of first service – arrival time)    |
| avg_turnaround_time | Mean over processes of (completion time – arrival time)           |
| avg_response_time   | Mean over processes of (first dispatch time – arrival time)       |
| cpu_utilization     | CPU busy time / total elapsed time (%)                            |
| throughput          | Processes completed per unit of simulated time                    |
| context_switches    | Number of running-PID changes (including switch-cost intervals)   |
| policy_switch_count | Decisions where the selected policy differs from the previous one |

## 8. Results and discussion

### 8.1 Held-out final test

All six methods are evaluated on the same 210 held-out workloads (master seed 19301) with quantum 4 and switch cost 1. Fixed policies and the heuristic are deterministic (210 rows each); Runtime Q-learning is evaluated greedily by all five trained models (1,050 rows) and averaged with equal weight per workload.

| Method (final test) | Mean waiting     | Mean turnaround  | Mean response   | CPU util %   | Throughput    | Ctx switches  | Policy switches |
|---------------------|------------------|------------------|-----------------|--------------|---------------|---------------|-----------------|
| FCFS                | 127.797 ± 79.81  | 146.612 ± 90.51  | 127.797 ± 79.81 | 92.24 ± 6.15 | 0.077 ± 0.056 | 14.00 ± 0.00  | 0.00 ± 0.00     |
| SJF                 | 93.345 ± 68.78   | 112.161 ± 79.39  | 93.345 ± 68.78  | 92.24 ± 6.15 | 0.077 ± 0.056 | 14.00 ± 0.00  | 0.00 ± 0.00     |
| Round Robin         | 215.729 ± 172.82 | 234.544 ± 183.45 | 26.220 ± 9.15   | 78.22 ± 1.90 | 0.068 ± 0.054 | 74.77 ± 40.70 | 0.00 ± 0.00     |
| Priority            | 129.207 ± 80.81  | 148.022 ± 91.53  | 129.207 ± 80.81 | 92.24 ± 6.15 | 0.077 ± 0.056 | 14.00 ± 0.00  | 0.00 ± 0.00     |
| Causal heuristic    | 203.404 ± 159.23 | 222.220 ± 169.85 | 40.070 ± 17.05  | 80.48 ± 2.22 | 0.071 ± 0.058 | 67.37 ± 39.49 | 3.21 ± 0.67     |
| Runtime Q-learning  | 104.789 ± 71.43  | 123.604 ± 81.86  | 86.341 ± 59.82  | 90.84 ± 7.04 | 0.076 ± 0.054 | 17.32 ± 4.66  | 4.73 ± 2.83     |

Table 8.1: Held-out final-test results, mean ± SD across the 210 workloads (Q-learning also averages 5 model seeds).

Held-out final test: 210 workloads (seed 19301), 5 Q-learning model seeds

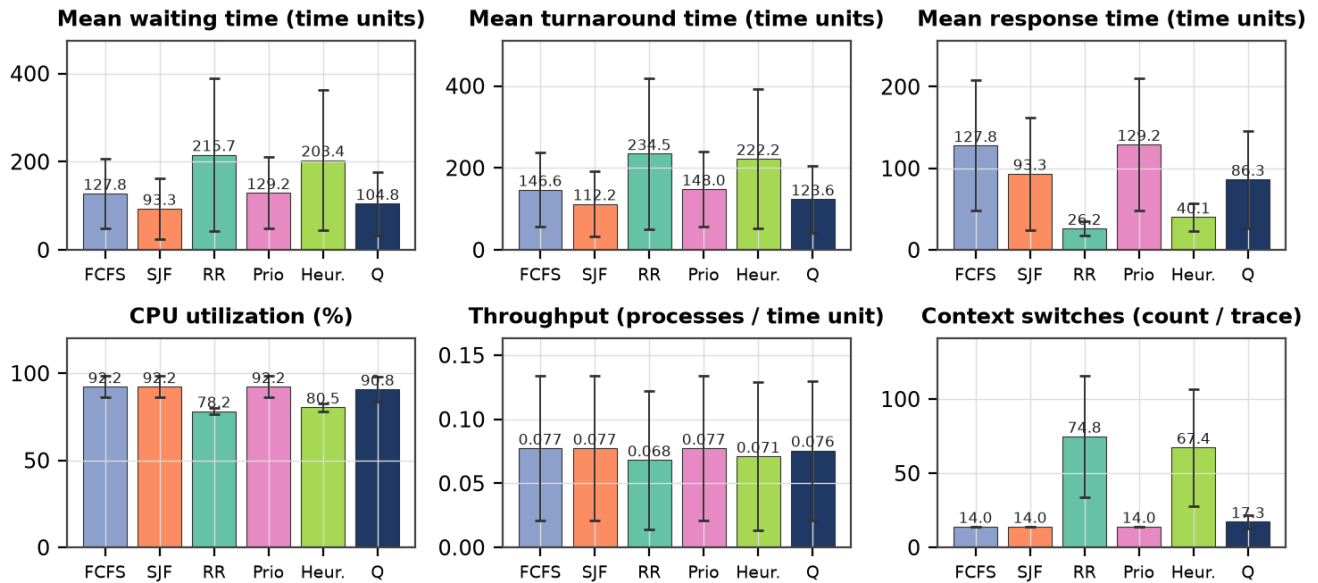


Figure 8.1: Six metrics on the 210 held-out workloads (error bars = SD).

### 8.2 Spread across workloads

Distribution across the 210 held-out workloads (whiskers = 1.5 IQR)

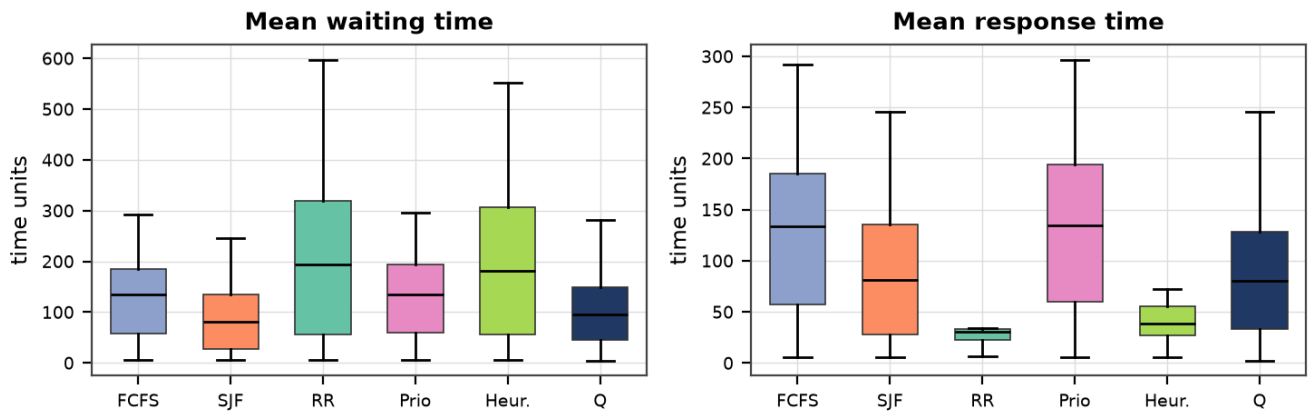


Figure 8.2: Per-workload distribution of waiting and response time. The medians show the same ordering as the means: the learned controller sits between SJF and the heuristic.

### 8.3 Per-family behaviour

| Family (n = 30)       | FCFS wait / resp | SJF wait / resp | RR wait / resp | Priority wait / resp | Heuristic wait / resp | Runtime Q wait / resp | Q switches |
|-----------------------|------------------|-----------------|----------------|----------------------|-----------------------|-----------------------|------------|
| short jobs            | 56.85 / 56.85    | 22.88 / 22.88   | 47.52 / 20.90  | 56.22 / 56.22        | 46.24 / 25.76         | 40.47 / 35.59         | 6.12       |
| long jobs             | 151.75 / 151.75  | 97.81 / 97.81   | 228.69 / 30.02 | 154.90 / 154.90      | 215.50 / 46.71        | 114.18 / 96.76        | 3.64       |
| mixed                 | 171.65 / 171.65  | 121.23 / 121.23 | 283.69 / 32.20 | 175.89 / 175.89      | 266.52 / 51.00        | 133.07 / 109.70       | 3.03       |
| cpu bursty            | 244.49 / 244.49  | 213.51 / 213.51 | 518.70 / 33.22 | 246.21 / 246.21      | 483.24 / 58.79        | 223.74 / 184.00       | 2.85       |
| priority skewed       | 183.06 / 183.06  | 131.16 / 131.16 | 308.26 / 31.76 | 184.45 / 184.45      | 287.98 / 48.61        | 142.25 / 118.59       | 3.00       |
| staggered interactive | 18.63 / 18.63    | 17.06 / 17.06   | 9.65 / 6.66    | 18.63 / 18.63        | 18.59 / 18.59         | 10.91 / 9.15          | 8.84       |
| poisson arrivals      | 68.14 / 68.14    | 49.76 / 49.76   | 113.58 / 28.78 | 68.15 / 68.15        | 105.77 / 31.03        | 68.91 / 50.59         | 5.63       |

Table 8.2: Per-family mean waiting / mean response (all six methods) and mean policy switches per trace for Runtime Q-learning.

Per-family means on the held-out final test (30 workloads per family; poisson\_arrivals was never trained on)

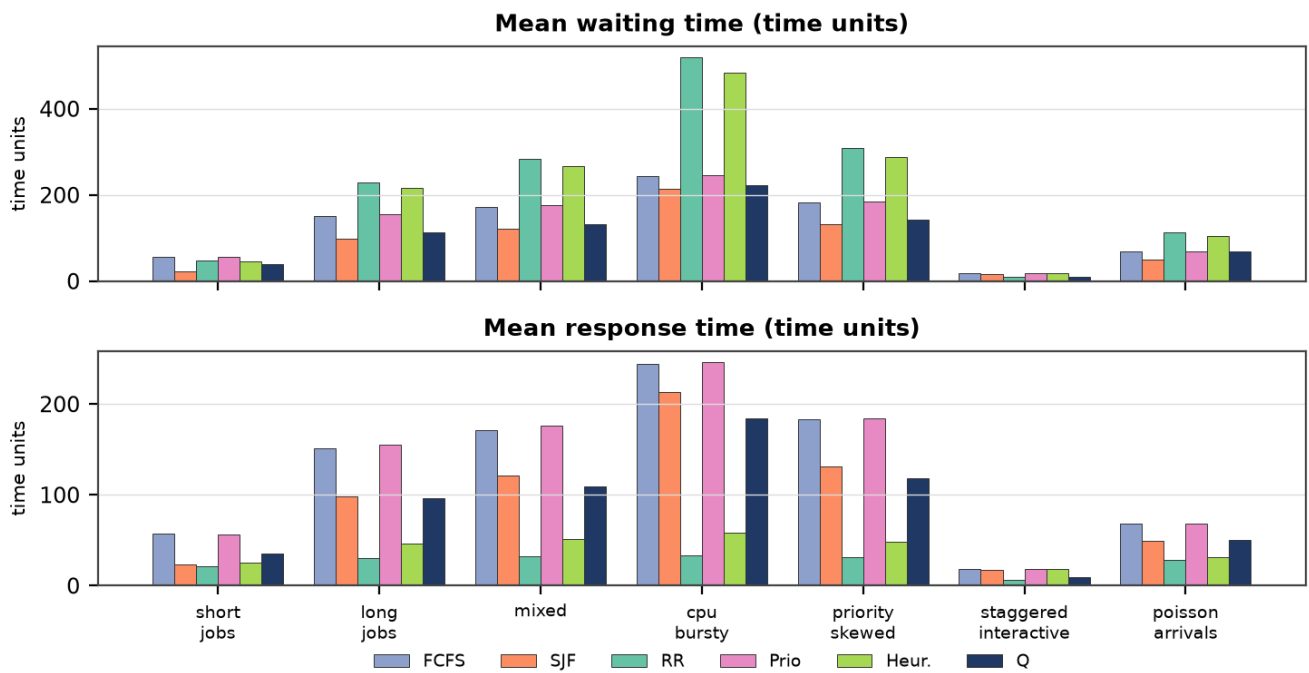


Figure 8.3: Per-family waiting and response time on the held-out final test. poisson\_arrivals was never seen during training.

## 8.4 Learning dynamics

5 independent agents x 1,200 sequential episodes (6,000 training workloads)

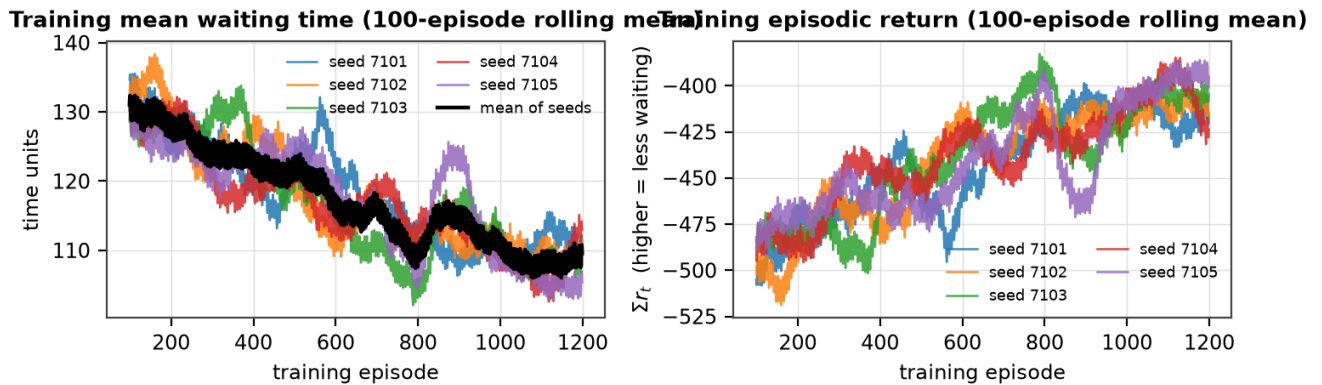


Figure 8.4: Sequential training: rolling mean waiting time and episodic return over 1,200 episodes for each of the five independent agents.

## 8.5 Paired comparisons with uncertainty

Paired differences are computed per workload (and per model seed for Q-learning) and resampled with a family-stratified crossed bootstrap (1,000 replicates, seed 104729). Positive values favour the reference; negative values favour the target.

| Target    | Reference | Metric           | Mean difference | 95% bootstrap CI    | Reading           |
|-----------|-----------|------------------|-----------------|---------------------|-------------------|
| Runtime Q | SJF       | waiting time     | +11.443         | [+5.022, +19.297]   | favours reference |
| Runtime Q | SJF       | response time    | -7.004          | [-9.541, -4.305]    | favours target    |
| Runtime Q | SJF       | context switches | +3.322          | [+2.209, +4.555]    | favours reference |
| Runtime Q | SJF       | CPU util %       | -1.402          | [-1.843, -1.005]    | favours reference |
| Runtime Q | Heuristic | waiting time     | -98.616         | [-106.097, -90.192] | favours target    |
| Runtime Q | Heuristic | response time    | +46.272         | [+43.472, +49.100]  | favours reference |

| Target    | Reference | Metric           | Mean difference | 95% bootstrap CI     | Reading               |
|-----------|-----------|------------------|-----------------|----------------------|-----------------------|
| Runtime Q | Heuristic | context switches | -50.050         | [-51.929, -48.066]   | favours target        |
| Runtime Q | Heuristic | CPU util %       | +10.354         | [+9.890, +10.834]    | favours target        |
| Heuristic | SJF       | waiting time     | +110.059        | [+106.219, +114.032] | favours reference     |
| Heuristic | SJF       | response time    | -53.276         | [-56.026, -50.873]   | favours target        |
| Heuristic | SJF       | context switches | +53.371         | [+51.666, +55.191]   | favours reference     |
| Heuristic | SJF       | CPU util %       | -11.756         | [-11.992, -11.518]   | favours reference     |
| RR        | SJF       | waiting time     | +122.383        | [+117.619, +127.315] | favours reference     |
| RR        | SJF       | response time    | -67.126         | [-70.219, -64.285]   | favours target        |
| RR        | SJF       | context switches | +60.767         | [+58.995, +62.491]   | favours reference     |
| RR        | SJF       | CPU util %       | -14.021         | [-14.241, -13.760]   | favours reference     |
| FCFS      | SJF       | waiting time     | +34.451         | [+32.640, +36.540]   | favours reference     |
| FCFS      | SJF       | response time    | +34.451         | [+32.605, +36.440]   | favours reference     |
| FCFS      | SJF       | context switches | +0.000          | [+0.000, +0.000]     | favours no difference |
| FCFS      | SJF       | CPU util %       | +0.000          | [+0.000, +0.000]     | favours no difference |

Table 8.3: Paired differences (target – reference) with family-stratified crossed-bootstrap confidence intervals on the final test set.

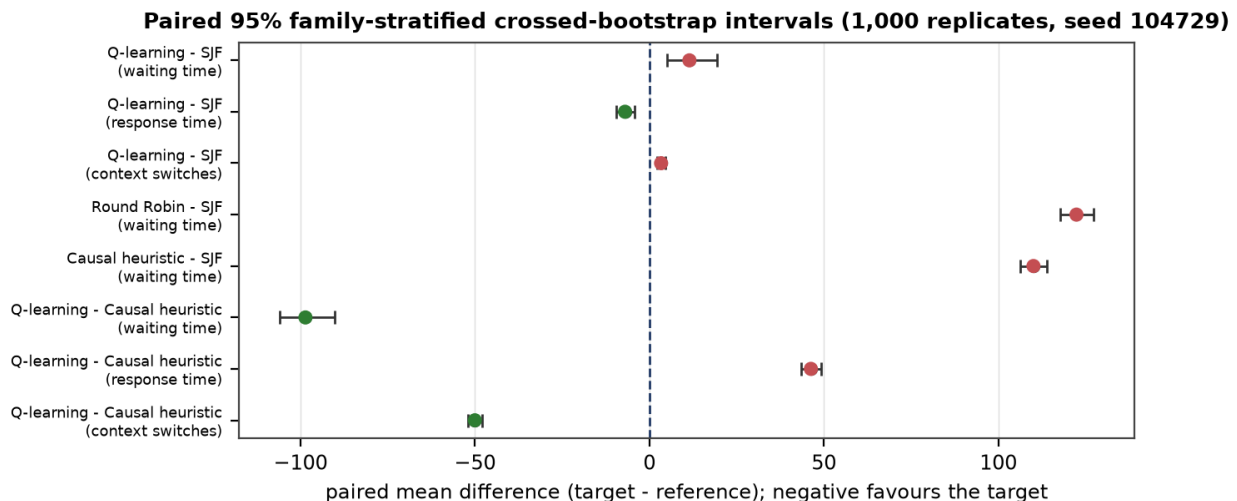


Figure 8.5: Forest plot of selected paired comparisons. Green markers favour the target, red markers favour the reference; bars are 95% bootstrap intervals.

## 8.6 Key findings

- **Adaptation beats Round Robin and the fixed non-SJF policies.** Runtime Q-learning waits 104.789 versus 215.729 for RR (51.4% lower), 127.797 for FCFS and 129.207 for Priority.
- **Response time improves over SJF.**  $Q - SJF = -7.004$  (95% CI [-9.541, -4.305]) — the learned controller keeps some of RR's responsiveness without paying all of RR's waiting cost.
- **Honest negative result: it does not beat SJF on waiting time.**  $Q - SJF = +11.443$  (95% CI [+5.022, +19.297]), i.e. SJF's waiting time is significantly lower. Non-preemptive SJF receives exact bursts on arrival and never incurs a mid-job switch cost, so on these generators it is a very strong reference; the switching overhead also costs Q-learning -1.40 percentage points of utilisation.
- **Learning beats the predeclared heuristic decisively on waiting time.**  $Q - \text{heuristic} = -98.616$  (95% CI [-106.097, -90.192]) with -50.05 context switches per trace — so the gain comes from RL, not merely from being adaptive.
- **But the heuristic wins on response time** ( $Q - \text{heuristic} = +46.272$ , 95% CI [+43.472, +49.100]) because it slices aggressively with RR. Neither method dominates the other; the objective determines the winner.
- **The held-out family generalises.** On poisson\_arrivals, which is excluded from training, the controller still lands between SJF and RR rather than collapsing to a single policy (Table 8.2).

- **Overhead is negligible relative to simulated time.** Mean controller cost is 13.69  $\mu$ s for observation plus 21.24  $\mu$ s for selection per decision; it is measured but never charged to the schedule.

## 8.7 Tabular coverage and fallback behaviour

| Train seed | Episodes | Decisions | States visited (of 162) | (s, a) visited (of 648) | Train wall / Q-update (ms) |
|------------|----------|-----------|-------------------------|-------------------------|----------------------------|
| 7101       | 1,200    | 23,511    | 81 (50.0%)              | 267 (41.2%)             | 2,022 / 424                |
| 7102       | 1,200    | 23,398    | 89 (54.9%)              | 272 (42.0%)             | 2,050 / 426                |
| 7103       | 1,200    | 23,596    | 87 (53.7%)              | 273 (42.1%)             | 1,977 / 417                |
| 7104       | 1,200    | 23,358    | 83 (51.2%)              | 276 (42.6%)             | 2,044 / 431                |
| 7105       | 1,200    | 23,671    | 88 (54.3%)              | 275 (42.4%)             | 2,042 / 432                |

Table 8.4: Per-seed tabular coverage, decisions and training time.

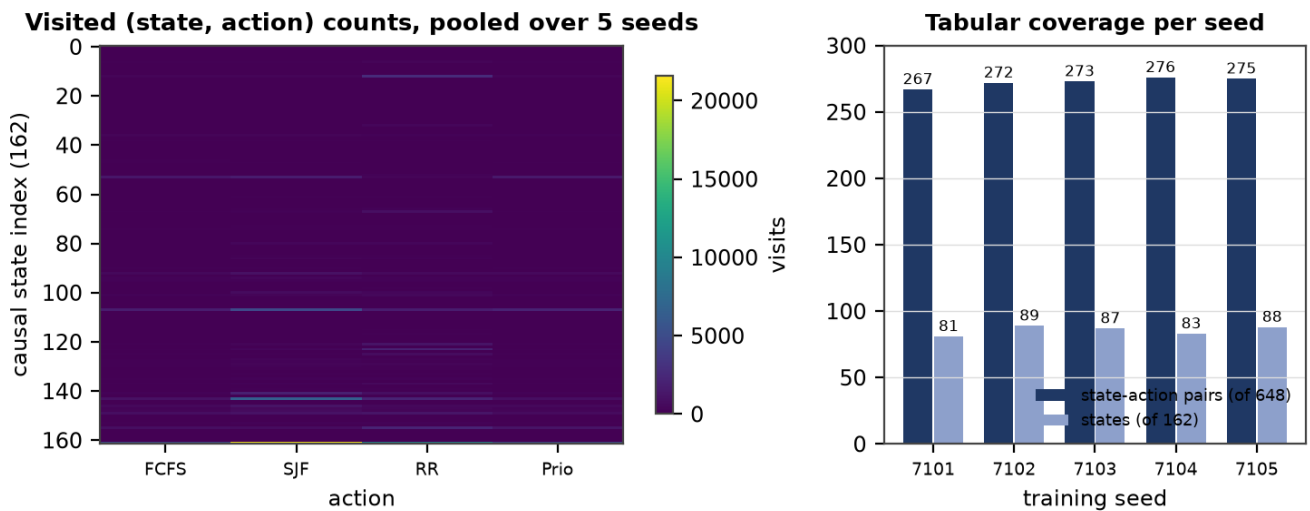


Figure 8.6: Learned state-action visitation pooled over the five seeds (left) and per-seed coverage of states and pairs (right).

## 8.8 Illustrative learned trace

The trace below is selected from the *validation* split by a predeclared rule (maximise policy switches among Q traces with zero unseen-state fallbacks): training seed 7103, family staggered\_interactive, repetition 0, fingerprint 275409d7458331f6. It contains 15 policy switches across 23 decisions with 5.933 mean waiting time and 2.933 mean response time. It is illustrative of the switching behaviour and explicitly **not representative** of average performance.

### Illustrative validation trace — seed 7103, staggered\_interactive rep 0 23 decisions, 15 policy switches (5.93 wait / 2.93 response)

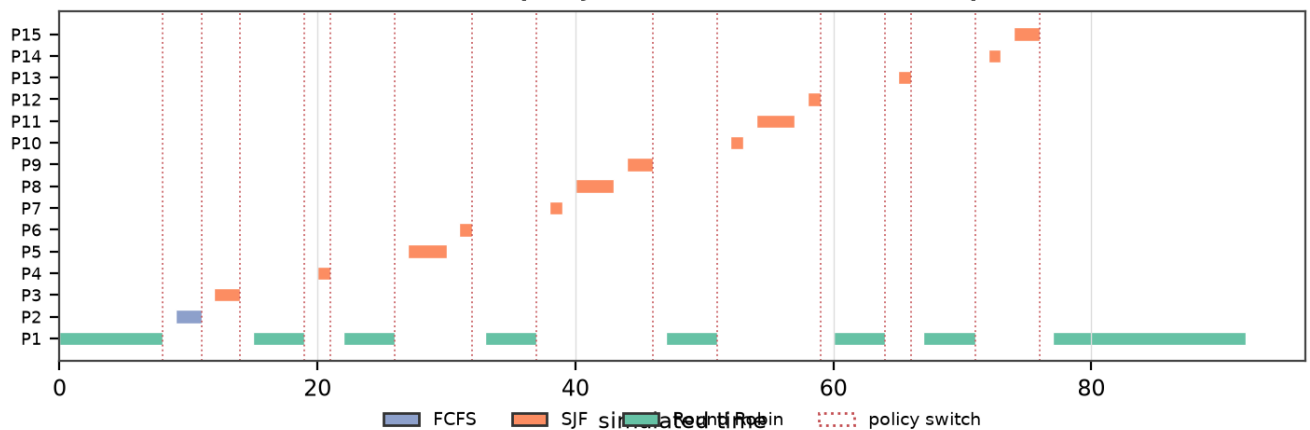


Figure 8.7: Illustrative validation trace. Each bar is one execution segment coloured by the policy chosen for it; dotted lines mark policy switches.

## 8.9 Validation versus held-out test

| Method             | Val waiting (140) | Test waiting (210) | Val response | Test response | Val ctx switches | Test ctx switches |
|--------------------|-------------------|--------------------|--------------|---------------|------------------|-------------------|
| FCFS               | 128.410           | 127.797            | 128.410      | 127.797       | 14.00            | 14.00             |
| SJF                | 95.797            | 93.345             | 95.797       | 93.345        | 14.00            | 14.00             |
| Round Robin        | 220.334           | 215.729            | 26.387       | 26.220        | 75.39            | 74.77             |
| Priority           | 129.847           | 129.207            | 129.847      | 129.207       | 14.00            | 14.00             |
| Causal heuristic   | 209.366           | 203.404            | 39.412       | 40.070        | 68.50            | 67.37             |
| Runtime Q-learning | 104.896           | 104.789            | 88.690       | 86.341        | 17.06            | 17.32             |

Table 8.5: Validation (master seed 8201, 140 workloads) versus held-out final test (master seed 19301, 210 workloads). The two splits agree closely, so no split-specific tuning is implied.

## 9. Testing and verification

Verification is automated with pytest. The suite covers the standalone schedulers, the workload models and metrics, the causal encoder and Q-learning update, the runtime experiment pipeline, and the legacy offline reference. The counts below were collected on the reporting host immediately before this report was generated.

| Test module                | Checks | What it protects                                                          |
|----------------------------|--------|---------------------------------------------------------------------------|
| test_adaptive.py           | 18     | Legacy offline selector: state encoding, decisions, read-only evaluation  |
| test_experiment_design.py  | 4      | Frozen seeds, split sizes and configuration invariants                    |
| test_experiments.py        | 30     | Legacy experiment pipeline, metrics and report generation                 |
| test_fcfs.py               | 11     | FCFS ordering, idle handling and metric computation                       |
| test_generator.py          | 7      | Workload family generation and reproducibility from seeds                 |
| test_metrics.py            | 14     | Waiting / turnaround / response / utilisation / throughput definitions    |
| test_priority.py           | 14     | Priority ordering, ties and non-preemption                                |
| test_q_learning.py         | 29     | Q updates, epsilon schedule, masking, visit counts, evaluation            |
| test_reward.py             | 16     | Reward composition and clipping against the reference policies            |
| test_round_robin.py        | 16     | Quantum boundaries, re-enqueue order and completion behaviour             |
| test_runtime_experiment.py | 4      | Split disjointness, determinism, artifact regeneration                    |
| test_runtime_scheduler.py  | 23     | Runtime environment: fixed-action equivalence, causality, reward identity |
| test_schedule_result.py    | 14     | Schedule trace validation and aggregate result objects                    |
| test_sjf.py                | 12     | SJF selection, tie-breaking and non-preemption                            |
| test_state.py              | 24     | Offline state discretisation and mixed-radix indexing                     |
| test_workload_models.py    | 37     | Process/workload models, validation and trace invariants                  |

Table 9.1: Automated test inventory (273 checks total).

### 9.1 Verification claims covered by the suite

- **Fixed-policy equivalence.** Running the event-driven simulator with a constant action reproduces the standalone FCFS, SJF, RR and Priority schedules exactly, including metrics, context switches and idle time.
- **Causal hidden-future invariance.** Changing arrivals, bursts and priorities of processes that have not yet arrived, while keeping the observed prefix fixed, leaves the observation and action prefixes bit-for-bit identical.
- **Reward-to-waiting identity.**  $\sum_t r_t$  equals  $-\text{total waiting time} / q$  for  $\gamma = 1$ ; the terminal transition omits the bootstrap; first-visit exploration, unvisited-action masking and read-only evaluation are asserted.
- **Split disjointness and reproducibility.** Zero SHA-256 workload fingerprint overlap across train / validation / test, and deterministic metric regeneration.

### 9.2 Test run output

```
$ python -m pytest -q
..... [ 79%]
..... [100%]
273 passed in 9.51s
```

## 10. Legacy offline policy selector (reference only)

Review 1 produced a whole-workload offline selector: it observed complete pre-execution workload features (81 states), chose one policy for the entire workload, and was trained with counterfactual rewards computed from four complete schedules with zero switch cost. It is preserved for continuity and is documented separately, because it is not a causal controller and must never be mixed into the runtime claims above.

| Offline method (210 workloads, switch cost 0) | Mean waiting        | Mean turnaround     | Mean response      | CPU util % | Ctx switches | Wait vs selector |
|-----------------------------------------------|---------------------|---------------------|--------------------|------------|--------------|------------------|
| FCFS                                          | 123.39 $\pm$ 80.71  | 142.34 $\pm$ 91.49  | 123.39 $\pm$ 80.71 | 97.84      | 14.00        | +28.1%           |
| SJF                                           | 88.10 $\pm$ 67.11   | 107.06 $\pm$ 77.80  | 88.10 $\pm$ 67.11  | 97.84      | 14.00        | -0.6%            |
| Round Robin                                   | 167.57 $\pm$ 135.06 | 186.52 $\pm$ 145.75 | 19.38 $\pm$ 8.22   | 97.84      | 75.40        | +47.1%           |
| Priority                                      | 122.22 $\pm$ 81.44  | 141.17 $\pm$ 92.25  | 122.22 $\pm$ 81.44 | 97.84      | 14.00        | +27.5%           |
| Offline Policy Selector (Q-Learning)          | 88.66 $\pm$ 67.89   | 107.61 $\pm$ 78.54  | 88.11 $\pm$ 68.47  | 97.84      | 15.53        | reference        |

Table 10.1: Legacy offline selector. Because the selector sees the whole workload and pays no switch cost, these numbers are not comparable with the runtime results and are reported only as a historical reference.

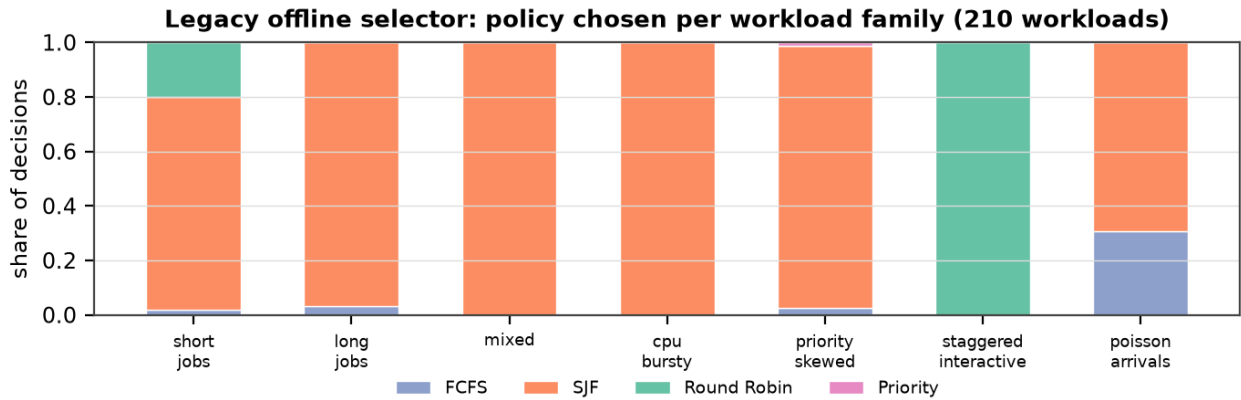


Figure 10.1: Legacy offline selector — share of each policy per workload family. It learned to choose SJF on batch-like families and RR on interactive ones.

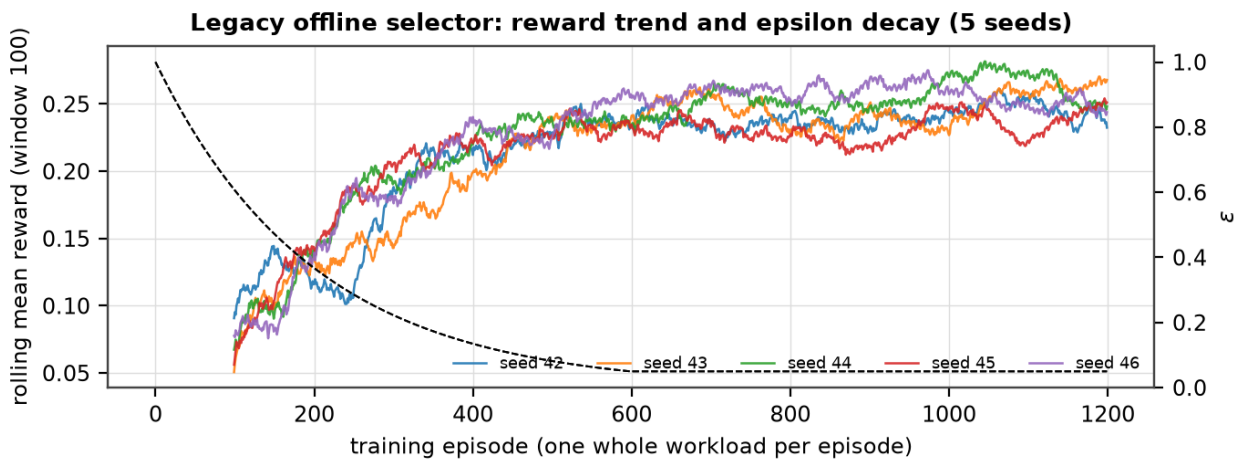


Figure 10.2: Legacy offline selector training: rolling mean reward (coloured) and epsilon decay (dashed black).

## 11. Limitations and future work

| Limitation                    | Current state                                                     | Planned direction                                                                                |
|-------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Synthetic workloads           | Seven generator families with known parameters; not trace-driven. | Replay real workload traces and add I/O bursts.                                                  |
| Burst knowledge               | Exact burst is known on arrival (SJF assumption).                 | Add a burst predictor with explicit prediction error.                                            |
| Single CPU                    | No multicore, migration cost or cache/bus contention.             | Extend the environment to N cores with per-core queues.                                          |
| No I/O or blocking            | Processes never block; the queue is CPU-bound only.               | Model I/O bursts and blocked/wake transitions.                                                   |
| Coarse discretisation         | 162 states; only ~42% of state-action pairs are ever visited.     | Tile coding, eligibility traces, or a small function approximator with the same causal contract. |
| Single objective              | The reward is waiting time only; response time is not optimised.  | Multi-objective / constrained RL balancing waiting, response and switch cost.                    |
| Fixed quantum and switch cost | $q = 4$ and cost = 1 throughout.                                  | Sensitivity study over quantum and switch cost; learn the quantum too.                           |
| Simulation only               | No kernel integration; timings are host-specific.                 | Prototype in a user-space scheduler or an eBPF-driven policy hook.                               |

The most important limitation to restate is the negative result: on these generators, the learned runtime controller does **not** beat non-preemptive SJF on mean waiting time. Because SJF is given exact bursts at arrival and never pays a mid-job switch cost, it is a demanding reference. The learned controller's measurable, reproducible gains are against Round Robin, FCFS, Priority and the predeclared adaptive heuristic, plus a response-time improvement over SJF.

## 12. Conclusion

Review 2 delivers a working, causally constrained, event-driven adaptive scheduler and an evaluation that is designed to survive scrutiny. The runtime environment preserves the semantics of the four classical policies while allowing the policy to change at every dispatch or quantum boundary; the controller sees only arrived work through a 162-state abstraction; and the learning objective is provably identical to minimising total waiting time because the undiscounted return equals  $-\text{WaitingTime} / q$ .

On 210 untouched held-out workloads the learned controller averages 104.789 time units of waiting time, against 215.729 for Round Robin, 127.797 for FCFS, 129.207 for Priority and 203.404 for the predeclared heuristic, while SJF remains the strongest waiting-time reference at 93.345. Every comparison is paired and reported with a family-stratified bootstrap interval; every artifact — Q values, visit counts, coverage, fallback counts, workload fingerprints and metric tables — is committed; and the entire experiment, this report included, regenerates from declared seeds with a single command.

The honest conclusion is therefore conditional rather than triumphal: runtime adaptation learned with a small causal state abstraction beats every fixed policy except SJF on waiting time, beats a hand-written adaptive rule by a wide margin, and improves response time relative to SJF — but it does not beat SJF on the metric it optimises, and the report says so. That is the result the experiment supports.

## References

- [1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts* — CPU scheduling chapter (FCFS, SJF, RR, Priority, multi-level feedback queues).
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems* — scheduling in batch, interactive and real-time systems.
- [3] A. Shrivastava, *Reinforcement Learning for Scheduling Threads on a Multi-Core Processor*, CS229 project report, Stanford University, 2010. [cs229.stanford.edu/proj2010/Shrivastava-ReinforcementLearningForSchedulingThreadsOnAMultiCoreProcessor.pdf](https://cs229.stanford.edu/proj2010/Shrivastava-ReinforcementLearningForSchedulingThreadsOnAMultiCoreProcessor.pdf)
- [4] G. Velingkar, J. K. Kumar, R. Varadarajan, S. Lanka and M. Anand Kumar, *Task Scheduling Using Deep Q-Learning*, Lecture Notes in Electrical Engineering 858, Springer, 2022. doi:10.1007/978-981-19-0840-8\_58
- [5] *Dynamic Operating System Scheduling Using Double DQN: A Reinforcement Learning Approach to Task Optimization*, arXiv:2503.23659, 2025. [arxiv.org/abs/2503.23659](https://arxiv.org/abs/2503.23659)
- [6] *A Context-Aware Intelligent Scheduling Framework for Modern Operating Systems*, Journal of Computational and Cognitive Engineering, 2026 (random-forest classification with a deep Q-network). doi:10.47852/bonviewJCCE62026664
- [7] T. Zhou and M. Lin, *CPU frequency scheduling of real-time applications on embedded devices with temporal encoding-based deep reinforcement learning*, Journal of Systems Architecture (arXiv:2309.03779), 2023. [arxiv.org/abs/2309.03779](https://arxiv.org/abs/2309.03779)
- [8] C. Shyalika, T. Silva and A. Karunananda, *Reinforcement learning in dynamic task scheduling: a review*, SN Computer Science 1, article 306, 2020.
- [9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* — temporal-difference learning,  $\epsilon$ -greedy exploration and tabular Q-learning.
- [10] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap* — paired bootstrap confidence intervals used for all comparisons in Section 8.

## Appendix A. Reproducibility and artifact index

### A.1 Commands

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.lock

python -m pytest -q                # 273 unit and integration tests
python main.py runtime-experiment  # reproduces results/runtime/ from the declared seeds
python main.py runtime-experiment --results-dir /tmp/os-project-runtime # isolated verification run
python main.py experiment          # reproduces the legacy offline selector

# Regenerate this PDF from the committed artifacts
pip install -r requirements-report.txt
python tools/build_review2_report.py
```

### A.2 Artifact index

| Path                                                          | Contents                                                                                 |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------|
| results/runtime/runtime_final_test_metrics.csv                | 2,100 rows — six metrics, switch and decision counts, timings, final test                |
| results/runtime/runtime_validation_metrics.csv                | 1,400 rows — the same columns for the validation split                                   |
| results/runtime/runtime_training_metrics.csv                  | 6,000 rows — sequential training outcomes per episode                                    |
| results/runtime/runtime_paired_comparisons.csv                | Paired differences with 95% crossed-bootstrap intervals                                  |
| results/runtime/runtime_state_action_coverage.csv             | Visited states/pairs, fallbacks, training and evaluation timings                         |
| results/runtime/runtime_q_table.csv                           | 3,240 rows — Q values and visit counts per seed                                          |
| results/runtime/runtime_workload_manifest.csv                 | 6,350 rows — split, seeds and workload fingerprints                                      |
| results/runtime/runtime_learned_switch_demo.json              | Illustrative validation trace: observations, actions, switch times, trace                |
| results/runtime/runtime_summary.json                          | Configuration, split fingerprints, all means, demo and software versions                 |
| results/runtime/runtime_{validation,final_test}_decisions.csv | Full causal event logs — generated by the CLI, intentionally gitignored, reproducible    |
| results/ (root)                                               | Legacy offline selector: metrics.csv, decisions.csv, summary.json, training_history.json |
| figures/                                                      | Legacy selector figures; figures/review2/ holds the figures in this report               |

### A.3 Environment used for the recorded run

| Item           | Value                                            |
|----------------|--------------------------------------------------|
| Python         | 3.11.2 (main, Apr 8 2026, 01:58:00) [GCC 12.2.0] |
| Platform       | Linux-6.1.158+-x86_64-with-glibc2.36             |
| NumPy / pandas | 2.4.6 / 3.0.6                                    |
| Matplotlib     | 3.11.2                                           |
| pytest         | 9.1.1                                            |